



Jenkins – Docker – GitHub Integration Scenario Explained

Stage-by-Stage Explanation

Stage 1 – Checkout Source Code

```
stage('Checkout Source Code') {  
  steps {  
    git branch: 'main', url: "${GIT_REPO}"  
  }  
}
```

Connects to GitHub

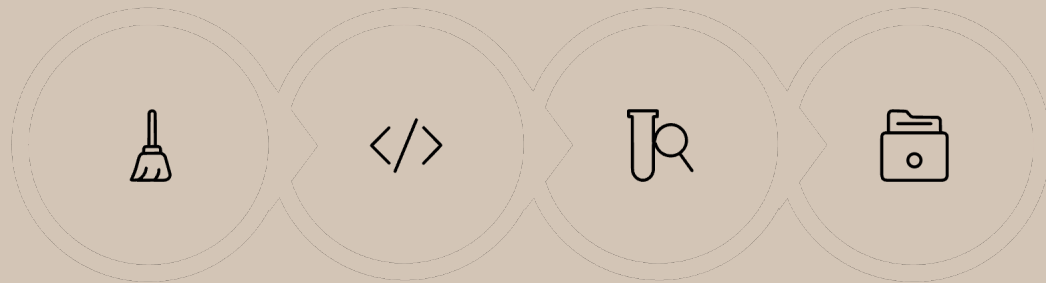
Downloads the latest source
code

Creates the Jenkins workspace

Stage 2 – Build Java Application

```
stage('Build Java Application') {  
  steps {  
    sh 'mvn clean package'  
  }  
}
```

Maven Lifecycle



Clean

Compile

Test

Package

Output

target/sample-java-app.jar



Stage 3 – Run Unit Tests

```
stage('Run Unit Tests') {  
  steps {  
    sh 'mvn test'  
  }  
}
```

Runs all JUnit tests before creating the Docker image.

Stage 4 – Build Docker Image

```
stage('Build Docker Image') {  
  steps {  
    sh """  
      docker build \\  
        -t sample-java-app:v1 \\  
        -t sample-java-app:latest .  
    """  
  }  
}
```

Creates two tags:

sample-java-app:v1

sample-java-app:latest

Stage 5 – Docker Login

```
withCredentials([
    usernamePassword(
        credentialsId: 'dockerhub-creds',
        usernameVariable: 'DOCKER_USER',
        passwordVariable: 'DOCKER_PASS'
    )
])
```

Jenkins securely retrieves the Docker Hub username and password.

Login command:

```
echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER" --password-stdin
```

No password is exposed in the console output.

Stage 6 – Push Docker Image

```
docker push sample-java-app:v1
```

```
docker push sample-java-app:latest
```

Uploads the image to Docker Hub.



Stage 7 – Stop Existing Container

```
docker stop sample-java-app || true
```

```
docker rm sample-java-app || true
```

→ Stops the old container if it exists.

→ Removes it before deployment.

→ `|| true` prevents the pipeline from failing if the container is absent.

Stage 8 – Deploy Container

```
docker run -d --name sample-java-app -p 8081:8080 sample-java-app:v1
```

Port Mapping



Stage 9 – Verify Deployment

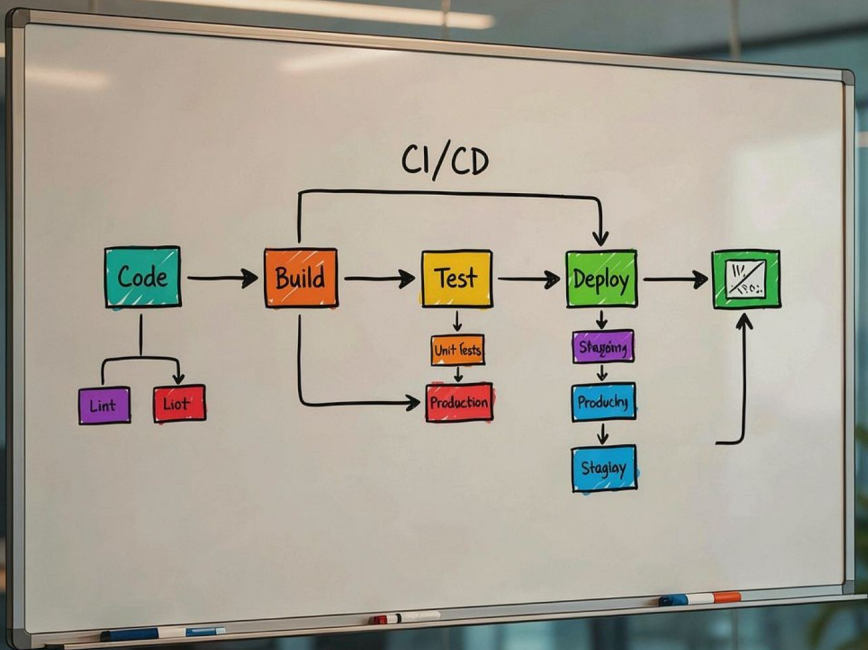
```
sleep 15
```

```
curl http://localhost:8081
```

Expected response:

Welcome to Sample Java Application

Application Running Successfully



Pipeline Execution Flow

Checkout

Build & Test

Containerize

Deploy & Verify

Jenkins Declarative Pipeline Best Practices

This pipeline follows Jenkins Declarative Pipeline best practices by using an environment block for reusable values, withCredentials for secure secret handling, separate stages for clear visibility, and post actions for cleanup and status reporting.



environment block

for reusable values



withCredentials

for secure secret handling



separate stages

for clear visibility



post actions

for cleanup and status reporting